PROJECT REPORT

ON

Student Resume Generator

Submitted by:
Shubham Gupta
UID: 26MCI10128

Submitted to:
Prof. Palwinder Kaur Mangat
(E9422)

Submitted in partial fulfillment of the requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

CHANDIGARH UNIVERSITY

University Institute of Computing

Punjab, India

BONAFIDE CERTIFICATE

This is to certify that the project report titled "**Student Resume Generator**" has been prepared as an academic project using Python, Tkinter, and other Python libraries. The project demonstrates graphical user interface (GUI) development, student information management, profile photo uploading and webcam capture, live resume preview, input handling, and PDF document generation.

The application enables students to enter their personal information, educational qualifications, technical skills, projects, work experience, certifications, and career objectives. It provides functionality to upload or capture a profile photograph, preview the resume, modify the entered information, and generate and download a professionally formatted resume in PDF format. The project utilizes Tkinter for the graphical user interface, Pillow and OpenCV for image processing and webcam capture, ReportLab for PDF generation, and PyMuPDF for displaying the resume preview within the application.

This project report is submitted in partial fulfilment of the requirements for the degree of Master of Computer Applications (MCA).

Student Signature: _____ Project Supervisor: _____

Head of Department: _____ Date: _____

CHAPTER 1: INTRODUCTION AND BACKGROUND STUDY

1.1 Project Overview

The Student Resume Generator is a desktop application developed in Python to simplify the creation of professional resumes. It lets a student enter personal information, educational qualifications, technical skills, projects, work experience, certifications and a career objective through a graphical user interface (GUI) built with Tkinter. The window is divided into two panels: a scrollable input form on the left and a live resume preview on the right.

The application addresses the common difficulties of manual resume formatting by organising the entered information into a fixed, well-structured layout. A profile photograph can either be uploaded from disk or captured from a webcam, and it is placed in the header of the resume together with the contact details.

1.2 Problem Definition

Students frequently need resumes for internships, campus placements and job applications. Creating a well-formatted resume manually in a word processor requires knowledge of document formatting and some design sense. The common difficulties are:

- Maintaining consistent fonts, spacing and alignment throughout the document.
- Arranging personal, educational and professional information in the correct order.
- Inserting, sizing and aligning a profile photograph.

The proposed system provides a single desktop-based solution that collects the student information, processes the photograph, generates a formatted PDF and displays a preview before the final document is saved.

1.3 Objectives and Scope

1.3.1 Objectives

1. To develop a user-friendly desktop application using Python and Tkinter.
2. To simplify the process of creating student resumes.
3. To collect personal, educational and professional information through a graphical interface.
4. To provide an option to upload a photograph or capture one using a webcam.
5. To generate a professionally structured resume in PDF format using ReportLab.

1.3.2 Scope of the Project

The application is intended for students, fresh graduates and job seekers who need a basic one-page resume for academic or professional purposes. It supports personal details, education, skills, projects, experience, certifications and a career objective, together with photograph upload, webcam capture, live preview and PDF download. Features such as multiple templates, database storage and a web version are outside the current scope and are listed as future enhancements in Chapter 4.

1.4 Technologies and Libraries Used

The project is written in Python and relies on a small number of standard and third-party libraries for its interface, image processing, PDF generation and file handling.

Table 1.1: Technologies and libraries used in the project

Technology / Library	Type	Purpose in the project
Python 3.9+	Language	Main programming language of the application

Tkinter	Standard library	Main window, frames, labels, entries, text boxes, buttons and canvas
ttk	Standard library	Themed widgets (used for the scrollbar of the input form)
Pillow (PIL)	Third party	Loading, resizing and displaying images; converts images for Tkinter
OpenCV (cv2)	Third party	Accessing the webcam and capturing photograph frames
ReportLab	Third party	Building and formatting the PDF resume (Platypus layout engine)
PyMuPDF (fitz)	Third party	Rendering PDF page 1 to a PNG image for the live preview
tempfile	Standard library	Creating temporary PDF files for previews and the webcam image
os / shutil / pathlib	Standard library	File removal, copying the preview PDF to the chosen location, path handling
filedialog	Standard library	Open-file dialog for photographs and Save-As dialog for the PDF
messagebox	Standard library	Information, warning, confirmation and error pop-ups

1.5 Existing System and Proposed Solution

Existing system. Traditional resume creation relies on word-processing software or online resume builders. The user must arrange the sections manually, adjust formatting, insert a photograph and repeatedly check the final document. Online builders may also require registration and upload personal data to a remote server.

Proposed solution. The Student Resume Generator provides an integrated, offline interface in which the user fills in a form, adds a photograph, watches the resume update live, makes corrections and saves the final PDF.

Table 1.2: Comparison of the existing and proposed systems

Existing system	Proposed system
Manual formatting is required	Formatting is handled automatically by the application
Section arrangement needs manual effort	Uses predefined resume sections in a fixed order
Photograph insertion may need separate editing	Supports photo upload and webcam capture (auto cropped to a square)
Preview needs another application	Live preview is displayed inside the application
Corrections need repeated re-formatting	Preview refreshes as the user types
File name must be typed manually	A file name is suggested from the student's name

CHAPTER 2: SYSTEM REQUIREMENTS AND DESIGN

2.1 Hardware and Software Requirements

The application requires a computer that can run Python and the libraries listed below. A webcam is optional and is needed only when the user wants to capture a photograph directly.

Table 2.1: Hardware and software requirements

Component	Requirement

Processor / RAM	Standard modern processor; 4 GB RAM or more recommended
Storage / Display	Space for Python, libraries and generated PDFs; monitor of 1050 x 700 or larger
Input devices	Keyboard and mouse; webcam optional (photograph capture)
Operating system	Windows or another compatible operating system
Language / GUI	Python 3.9 or later; Tkinter (bundled with Python on Windows)
Libraries	Pillow and OpenCV (image processing), ReportLab (PDF), PyMuPDF (preview)
Code editor	Visual Studio Code, PyCharm or another Python editor

The libraries can be installed with `pip install reportlab pillow opencv-python pymupdf`. Webcam capture additionally depends on a working camera and the operating-system camera permissions.

2.2 System Architecture

The application follows a modular, layered structure in which each layer performs a specific task. The graphical interface collects information, the processing logic organises it, and the relevant libraries handle photographs, PDF creation and preview rendering.

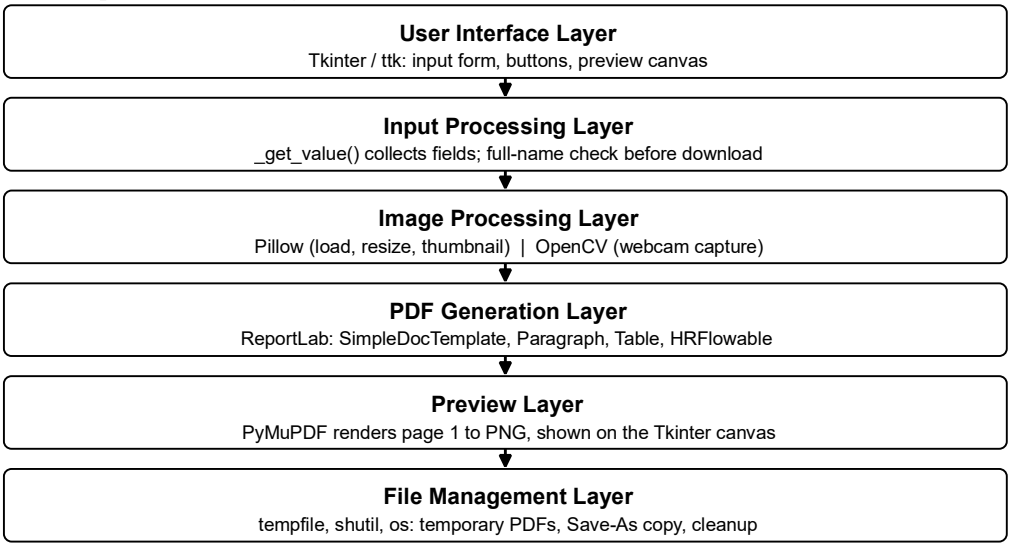


Figure 2.1: Layered architecture of the Student Resume Generator

2.3 Data Flow and Program Workflow

The program follows a sequence of operations that begins with information entry and ends with the creation of the final PDF resume, as shown in Figure 2.2.

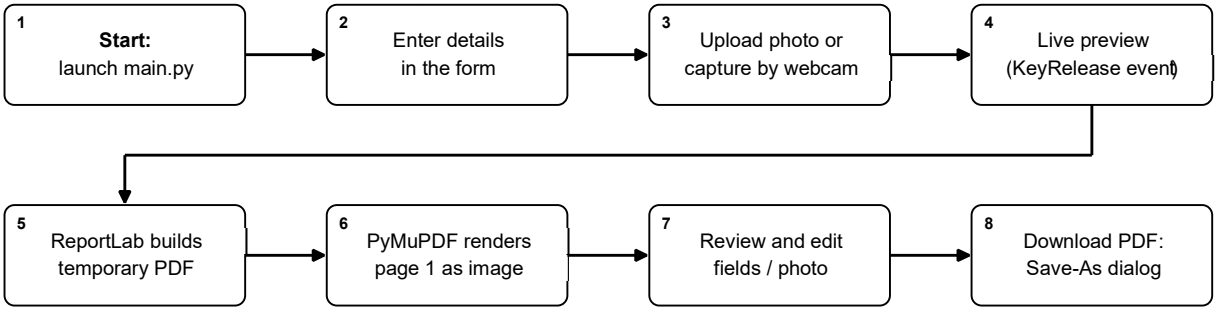


Figure 2.2: Program workflow from launch to PDF download

2.4 Student Information Management

The application organises student information into predefined fields. The values are read from the widgets into a dictionary keyed by field name and are passed to the PDF generator. Table 2.2 lists the fields; fields marked with an asterisk in the interface are required, although the current version enforces only the full name.

Table 2.2: Input fields of the application

Category	Field key	Label in the form	Widget
Personal	full_name	Full Name *	Entry
Personal	email	Email *	Entry
Personal	phone	Phone Number *	Entry
Personal	address	Address	Entry
Profile	linkedin	LinkedIn / Portfolio	Entry
Education	course	Course / Degree *	Entry
Education	college	College / University *	Entry
Education	graduation_year	Passing Year	Entry
Education	percentage	Percentage / CGPA	Entry
Skills	skills	Technical Skills *	Entry
Projects	projects	Projects	Text (3 lines)
Experience	experience	Experience / Internship	Text (3 lines)
Certifications	certifications	Certifications	Entry
Objective	objective	Career Objective *	Text (4 lines)

2.5 Functional and Non-Functional Requirements

Table 2.3: Functional requirements

ID	Requirement
FR1	The application shall provide a graphical interface for entering student details.
FR2	The application shall allow the user to upload a photograph (JPG, JPEG, PNG or WEBP).
FR3	The application shall support webcam-based photograph capture.
FR4	The application shall generate the resume in PDF format.
FR5	The application shall display a live preview of the generated resume.
FR6	The application shall allow users to modify information and refresh the preview.
FR7	The application shall check that the full name is entered before downloading.
FR8	The application shall let the user clear the form after confirmation.

2.5.1 Non-Functional Requirements

- **Usability:** the interface should be easy to understand and operate.

- **Performance:** previews should be generated and displayed within a reasonable time.
- **Reliability:** normal user operations should not terminate the application unexpectedly.
- **Portability and privacy:** runs wherever Python and the libraries are installed; all information is processed locally; there is no cloud storage or online profile management.

CHAPTER 3: IMPLEMENTATION AND APPLICATION DEVELOPMENT

3.1 Program Structure and Object-Oriented Design

The application is implemented in a single Python file, *main.py* (about 500 lines). All functionality is organised inside the `StudentResumeApp` class, whose constructor receives the Tkinter root window. The state of the application is held in a few instance attributes: `fields` (a dictionary of widgets), `profile_image_path`, `preview_files` (a set of temporary files to delete on exit), `current_temp_pdf` and `preview_tk_img`.

Table 3.1: Methods of the StudentResumeApp class

Method	Responsibility
__init__	Creates the main window, initialises state, builds the interface and renders the first preview
_build_ui	Builds the header, scrollable form, preview panel, buttons and bottom bar
_set_value / _get_value	Write or read a value for Entry and Text widgets through one interface
upload_photo	Opens a file dialog and stores the selected photograph path
_display_thumbnail	Shows a 60 x 60 thumbnail of the photograph in the form
open_webcam_dialog	Opens a live camera window and saves the captured frame as a JPEG
clear_form	Clears all fields after confirmation and restores default values
update_preview	Builds a temporary PDF, renders page 1 and draws it on the canvas
download_resume	Validates the name, shows the Save-As dialog and copies the PDF
close_app	Deletes tracked temporary files and closes the window
_create_pdf	Builds the formatted resume using ReportLab Platypus flowables

Listing 3.1 shows the import section and the constructor. The constructor registers `close_app` as the window-close handler so that temporary files are always cleaned up.

Listing 3.1: Imports and constructor of StudentResumeApp (main.py)

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import os
import shutil
import tempfile
import webbrowser
from pathlib import Path
from PIL import Image, ImageTk
import cv2
import pymupdf as fitz  # Updated PyMuPDF import
from reportlab.lib import colors
from reportlab.lib.enums import TA_LEFT, TA_CENTER
from reportlab.lib.pagesizes import A4
```

```
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, HRFlowable, Image as RLImage
)
from xml.sax.saxutils import escape

class StudentResumeApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Student Resume Generator - Pro Edition")
        self.root.geometry("1250x800")  # Fixed geometry specifier
        self.root.minsize(1050, 700)
        self.root.configure(bg="#eef3f8")

        self.fields = {}
        self.profile_image_path = None
        self.preview_files = set()
        self.current_temp_pdf = None
        self.preview_tk_img = None

        self.root.protocol("WM_DELETE_WINDOW", self.close_app)
        self._build_ui()
        self.update_preview()  # Initial render
```

3.2 Graphical User Interface Development

Tkinter provides the widgets for the desktop interface. The window is 1250 x 800 pixels with a minimum size of 1050 x 700. A dark blue header carries the application title.

Listing 3.2: Dynamic creation of form fields (inside _build_ui)

```
row = tk.Frame(form, bg="white")
row.pack(fill="x", pady=4)
tk.Label(
    row, text=label, width=20, anchor="w",
    font=("Segoe UI", 9, "bold"), bg="white", fg="#263648"
).pack(side="left", anchor="nw", pady=4)

if key in ("projects", "experience", "objective"):
    widget = tk.Text(
        row, height=3 if key != "objective" else 4,
        font=("Segoe UI", 10), wrap="word",  # Integer font size
        relief="solid", bd=1, padx=6, pady=4
    )
    widget.pack(side="left", fill="x", expand=True)
else:
    widget = tk.Entry(
        row, font=("Segoe UI", 10), relief="solid", bd=1  # Integer font size
    )
    widget.pack(side="left", fill="x", expand=True, ipady=5)

widget.bind("<KeyRelease>", lambda e: self.update_preview())
self.fields[key] = widget
```

3.3 Photo Uploading and Webcam Integration

Uploading. `upload_photo` opens a file dialog restricted to JPG, JPEG, PNG and WEBP files. Pillow opens the chosen image, creates a 60 x 60 thumbnail for the form, and the path is stored for use by ReportLab when the PDF is generated.

Webcam capture. `open_webcam_dialog` creates a modal window and opens the default camera with OpenCV. A timer calls `update_frame` every 30 ms; each frame is converted from BGR to RGB, cropped to a centred square, resized to 300 x 300 pixels and drawn on a canvas. When the user presses **Capture & Use Photo**, the current frame is saved as *webcam_capture.jpg* in the system temporary folder, becomes the profile photograph, the camera is released and the preview is refreshed. If the camera cannot be opened, an error message is shown (Listing 3.3).

Listing 3.3: Saving the captured webcam frame (inside open_webcam_dialog)

```
 def capture_and_save():
     if captured_frame[0] is not None:
 temp_dir = tempfile.gettempdir()
         photo_path = os.path.join(temp_dir,
 "webcam_capture.jpg")          img_pil =
 Image.fromarray(captured_frame[0])
 img_pil.save(photo_path)          self.profile_image_path
 = photo_path          self._display_thumbnail(photo_path)
 self.update_preview()      cap.release()
 dialog.destroy()
```

3.4 Live Resume Preview

The `update_preview` method reads every field into a dictionary, creates a temporary PDF with `tempfile.mkstemp` and calls `_create_pdf`. PyMuPDF then opens the file, loads page 0 and renders it to a PNG pixmap at 110 DPI. Pillow shrinks the image to fit the canvas and `ImageTk.PhotoImage` displays it. Every temporary file is recorded in `preview_files`, and the most recent one in `current_temp_pdf`, so that the download step can simply copy the file that the user has just seen.

Listing 3.4: Preview generation and rendering (update_preview)

```
def update_preview(self):
        data = {key: self._get_value(key) for key in self.fields}

        fd, temp_path = tempfile.mkstemp(suffix=".pdf")
        os.close(fd)

        try:
            self._create_pdf(data, temp_path)
            self.current_temp_pdf = temp_path
            self.preview_files.add(temp_path)

            doc = fitz.open(temp_path)
            page = doc.load_page(0)
            pix = page.get_pixmap(dpi=110)
            img_bytes = pix.tobytes("png")
            doc.close()

            from io import BytesIO
            img_pil = Image.open(BytesIO(img_bytes))

            canvas_w = self.preview_canvas.winfo_width()
            canvas_h = self.preview_canvas.winfo_height()
            if canvas_w < 50: canvas_w = 400
            if canvas_h < 50: canvas_h = 600

            img_pil.thumbnail((canvas_w, canvas_h))
            self.preview_tk_img = ImageTk.PhotoImage(img_pil)

            self.preview_canvas.delete("all")
            self.preview_canvas.create_image(
                canvas_w // 2, canvas_h // 2, image=self.preview_tk_img, anchor="center"
            )
        except Exception as e:
            print(f"Preview render notice: {e}")
```

3.5 Resume Formatting and PDF Generation

ReportLab Platypus builds the resume on an A4 page with margins of 0.55 inch (left and right) and 0.5 inch (top and bottom). PDF is convenient for sharing and printing because it preserves the layout across viewers. Custom paragraph styles give the resume a consistent look (Table 3.2), and all user text is passed through `xml.sax.saxutils.escape` so that characters such as < or & cannot break the layout.

The resume sections appear in this order: header (name, contact details and optional 0.85 x 0.85 inch photograph in a two-column table), Career Objective, Education (a small table), Technical Skills, Projects, Experience / Internship and Certifications, followed by a footer line. Empty sections are skipped.

3.6 Download, Input Validation and Temporary File Management

When the user clicks **Download PDF**, `download_resume` reads the fields and checks that the full name is not empty; if it is empty, a warning is displayed and no file is written. The suggested file name is built from the characters of the name that are letters, digits, spaces, underscores or hyphens, with spaces replaced by underscores and the suffix *_Resume.pdf*. After the user selects a location, `shutil.copyfile` copies the preview PDF and a success or error message is shown.

Listing 3.5: Download with validation, and cleanup on exit

```
def download_resume(self):
        data = {key: self._get_value(key) for key in self.fields}
        if not data["full_name"]:
            messagebox.showwarning("Missing Name", "Please enter at least your Full Name to
download.")
            return

        safe_name = "".join(ch for ch in data["full_name"] if ch.isalnum() or ch in (" ", "_", "-
")).strip()
        default_filename = (safe_name.replace(" ", "_") or "Student") + "_Resume.pdf"

        filepath = filedialog.asksaveasfilename(
            title="Download Resume PDF",
            defaultextension=".pdf",
            initialfile=default_filename,
            filetypes=[("PDF files", "*.pdf")]
        )
        if not filepath:
            return
        try:
            shutil.copyfile(self.current_temp_pdf, filepath)
            messagebox.showinfo("Success", f"Resume downloaded successfully!\n\nSaved
to:\n{filepath}")
        except OSError as exc:
            messagebox.showerror("Error", f"Could not download resume.\n\nDetails: {exc}")
```

CHAPTER 4: TESTING, RESULTS AND CONCLUSION

4.1 Testing Methodology

The application is tested manually by entering different combinations of student details, uploading photographs, capturing webcam images, observing the live preview, and downloading the final PDF. Each test case in Table 4.1 states the action, the expected result and a column in which the observed outcome is recorded after execution.

4.2 Test Cases and Validation

Table 4.1: Test cases

ID	Test case	Input or action	Expected result	Status
TC01	Launch application	Run *python main.py*	Main window opens with an initial preview	pass
TC02	Enter details	Fill in the form fields	Information is accepted and shown in the preview	pass
TC03	Upload photograph	Select a valid image file	Thumbnail is shown and photo appears in the resume header	pass
TC04	Capture photograph	Use the webcam option	Photo is captured if a camera is available	pass

TC05	Camera not available	Open webcam with no camera	"Webcam Error" message is displayed	pass
TC06	Refresh preview	Click Refresh Preview	Resume preview is regenerated and displayed	pass
TC07	Edit information	Modify fields	Updated information appears in the preview	pass
TC08	Missing full name	Download without a name	"Missing Name" warning is displayed	pass
TC09	Download resume	Enter a name and save	PDF is saved at the chosen location	pass

Note: these are the planned test cases. The Status column should be filled with Pass / Fail after running each test on the completed application.

4.3 Results and Analysis

The form allows users to enter information into predefined fields instead of designing the whole document manually. ReportLab converts the information into a structured PDF resume, and the photograph feature supports both image upload and webcam capture. The live preview lets users inspect the generated page before saving, which is useful for correcting spelling, updating skills, changing educational details or modifying the career objective. The following screenshots document the working application.

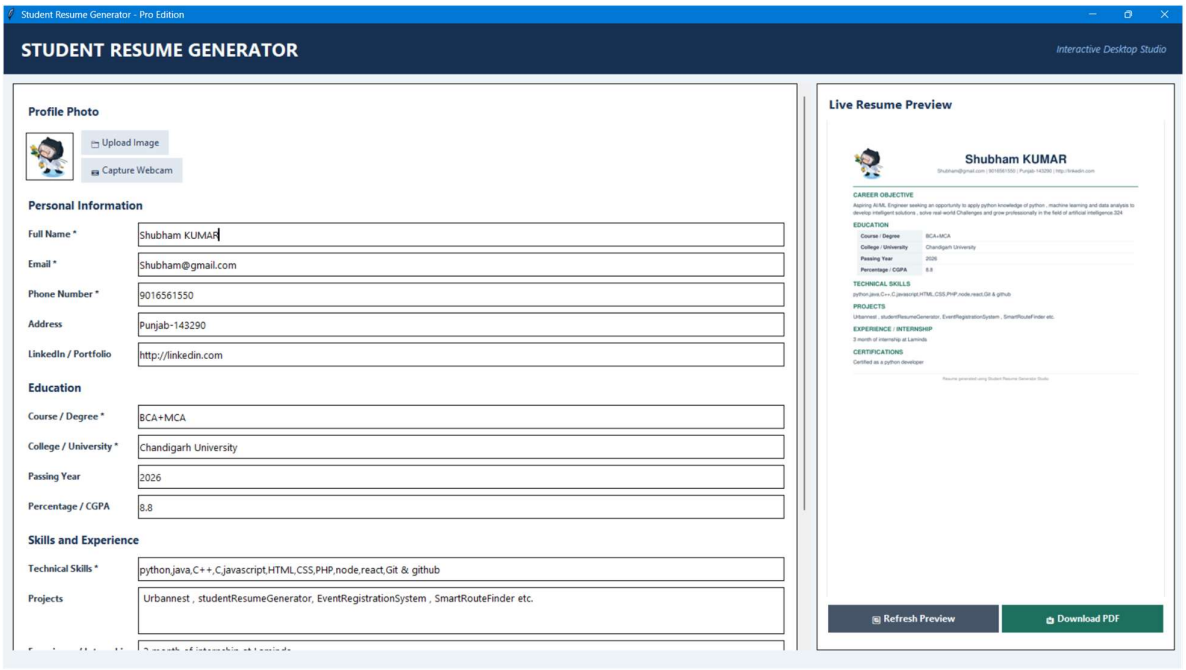


Figure 4.4: Final resume PDF opened in a PDF viewer after download

4.4 Limitations and Future Scope

Table 4.2: Current limitations

Limitation	Description
Single template	A predefined layout is used instead of a collection of selectable templates.
No database storage	Student profiles cannot be saved permanently and reopened later.

Basic validation	Only the full name is checked; email and phone are not validated.
Desktop only	Designed as a desktop program rather than a browser-based service.
Library dependencies	Python and the required libraries must be installed before running.
Single-page preview	Only the first page of the PDF is rendered in the preview.

4.4.1 Future Enhancements

- **Multiple resume templates:** professional, academic, modern and creative layouts.
- **Database integration:** store and retrieve student profiles with SQLite or MongoDB.
- **Improved validation:** email-format and 10-digit phone checks with clear error messages.
- **Customisable design:** choice of fonts, colours, spacing and section order.
- **Web-based version:** convert the application into a browser-based resume builder.
- **AI-based suggestions:** optional help for career objectives and project descriptions.
- **Multiple profiles:** separate resumes for different job roles; export to DOCX.

4.5 Conclusion

The Student Resume Generator shows how a practical desktop tool can be built with Python. Tkinter provides the interface, Pillow and OpenCV support photograph handling, ReportLab generates the final PDF and PyMuPDF displays the live preview. Users can enter their information, review the resume, make corrections and download the final document without formatting every section by hand.

The project demonstrates object-oriented design, GUI development, event handling, image processing, file handling and PDF generation. It is a useful starting point for automating resume creation, and future enhancements such as multiple templates, stronger validation, database storage and AI-based assistance can make it more flexible.

4.6 Pushing the Project to GitHub

GitHub is a platform for storing and managing source code with the Git version-control system. Publishing the project on GitHub preserves the source code, tracks changes and makes it easy to share with faculty members and other developers.

4.6.1 Project Folder Structure

Listing 4.1: Repository layout

```
Python_Mini_Project/
|-- main.py            # GUI, preview, PDF generation
(StudentResumeApp)
|-- requirements.txt   # reportlab, pillow, opencv-python,
pymupdf
|-- README.md          # overview, features and run instructions
`-- screenshots/       # optional: application and resume
                                screenshots
```

Table 4.3: Repository files

File	Description
main.py	Complete source code of the application
requirements.txt	External Python libraries required to run the project
README.md	Project overview, features, requirements and how to run
screenshots/	Optional folder containing images of the interface and the generated resume

4.6.2 GitHub Repository Link

Repository: https://github.com/Shubhamm-9016/Python_Mini_Project.git

4.6.3 Git Commands Used

git init
git add main.py requirements.txt README.md
git commit -m "Add Student Resume Generator project"
git branch -M main
git remote add origin https://github.com/Shubhamm-9016/Python_Mini_Project.git
git push -u origin main

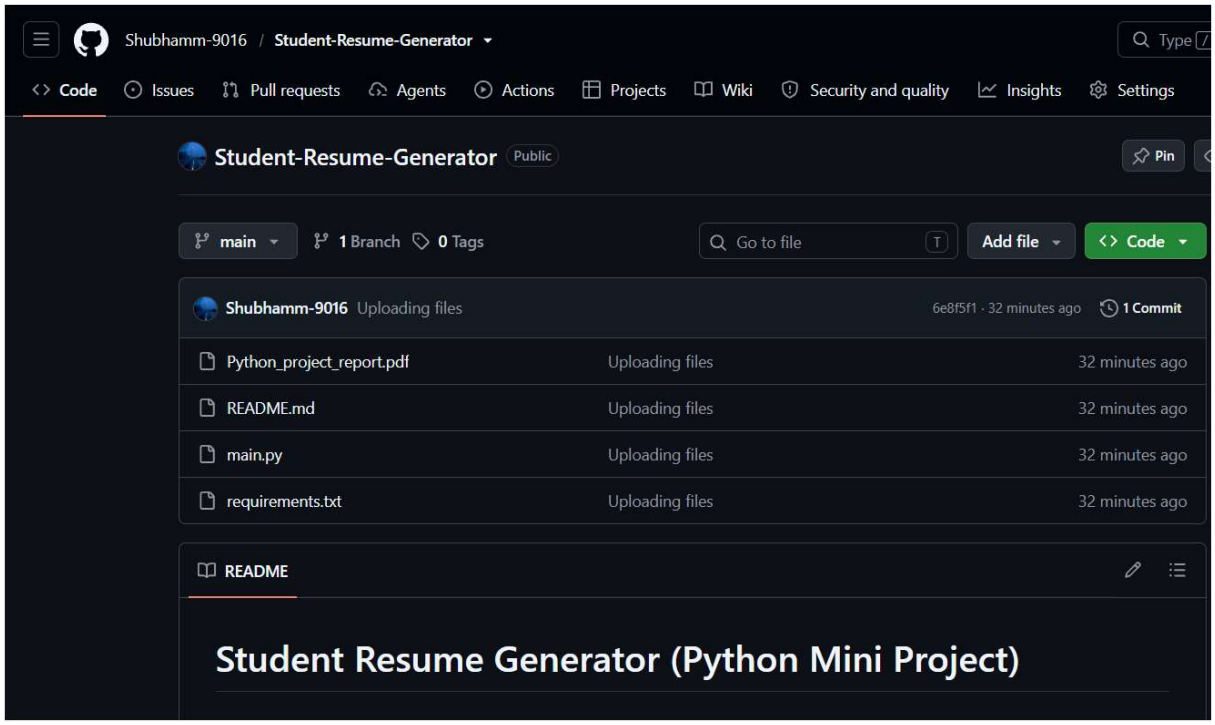


Figure 4.5: GitHub repository home page (repository name, description and file list)

REFERENCES

1. Python Software Foundation. *Python Documentation.* https://docs.python.org/3/
2. Python Software Foundation. *Tkinter — Python Interface to Tcl/Tk.* https://docs.python.org/3/library/tkinter.html
3. Pillow Contributors. *Pillow Documentation.* https://pillow.readthedocs.io/
4. OpenCV. *OpenCV Documentation.* https://docs.opencv.org/
5. ReportLab. *ReportLab Documentation.* https://docs.reportlab.com/
6. PyMuPDF. *PyMuPDF Documentation.* https://pymupdf.readthedocs.io/
7. Git. *Git Documentation.*
8. https://git-scm.com/doc
9. GitHub. *GitHub Documentation.* https://docs.github.com/